FarmwiseAI

CAMPUS PRODUCT CHALLENGE

Product Challenge Brief and Proposal Guidelines

Six real-world product challenges across Agentic AI, GeoAI, crop intelligence, urban vision and unified government intelligence

Ready to take on a challenge?

Register your team and pick your task on the enrollment platform.

CLICK HERE TO APPLY →

PROPOSAL AND SHORTLISTING BRIEF

Six challenges · Agentic AI · GeoAI · Crop Intelligence

Contents

How to Use This Brief 2

Proposal Submission Requirements 2

THE SIX CHALLENGES

TASK 1
Multi-Model, Multimodal and Agentic AI System 4

TASK 2
Multimodal GeoAI and Land Document Intelligence Platform 8

TASK 3
Satellite-Embedding-Based Crop Classification 11

TASK 4
Mobile-Image Crop, Stage and Disease Model 14

TASK 5
Google Street View-Based Urban Asset and Property Intelligence 17

TASK 6
Collector's Unified Application, Web and District Intelligence System 20

After Shortlisting: Prototype Development Stage 23

Ready to take on a challenge?

CLICK HERE TO REGISTER →

How to Use This Brief

This brief is for all participants. Each participant or team must select one challenge and submit a proposal describing what they intend to build. FarmwiseAI will review the proposals and shortlist the teams whose ideas, approach and project plans are the most promising and achievable.

> **Proposal stage:** Shortlisting will be based on the submitted proposal. A working prototype or product demonstration is not required at this stage. The task descriptions and examples explain the intended challenge and the capabilities that the proposed solution should address if the team is shortlisted.

Shortlisted teams will receive the required data package, technical inputs and approved cloud or model credits, and will then develop the proposed prototype over approximately two to three weeks.

Domain Knowledge Is Not a Prerequisite

> **Important:** Proposals will be assessed on the team's problem understanding, proposed capabilities, solution approach, architecture, innovation and feasibility. Prior knowledge of agriculture, land administration, districts, taluks, villages or government departmental processes is not required at the proposal stage.

The challenge descriptions use real FarmwiseAI application contexts so that proposed solutions address meaningful problems. Participants may focus on the technical capabilities to be developed rather than assuming prior expertise in the application domain.

Proposal Submission Requirements

The initial proposal should clearly explain the proposed solution and allow FarmwiseAI to assess whether a complete working prototype can reasonably be developed within approximately two to three weeks. A prototype is not expected at the proposal stage.

The Proposal Must Include

- Chosen challenge, proposed project title and project objective
- The team's understanding of the problem in its own words
- What the team proposes to build and the outcome it intends to demonstrate
- Proposed solution approach, system architecture, models, tools and technology stack
- Main components and capabilities that will be implemented in the prototype

- Any innovative, novel or exploratory idea that the team proposes to test
- Datasets, maps, documents, APIs, cloud resources and model access expected to be required
- A project plan for the two-to-three-week prototype-development period, showing the sequence of work, milestones and intended deliverables
- Team members and the responsibilities proposed for each member

> **Project plan:** The work plan should be realistic for the selected challenge. It should explain how the team will move from initial data understanding and architecture setup to implementation, integration, validation and the final working demonstration. Teams may define their own milestones and development sequence.

TASK 1

Multi-Model, Multimodal and Agentic AI System

Build a reusable AI architecture that plans a request, routes each step to the right model or tool and verifies the final result.

Objective

Build a generic agentic AI system that can accept text, documents, images and structured data; understand a request; create an execution plan; route individual steps to the most suitable model or tool; combine the outputs; verify important results; and recover when a model or tool fails. The architecture should be reusable and should not be designed only for one hard-coded query or domain.

Required Multi-Model Architecture

- A planner or orchestrator that decomposes the user request into clear steps
- At least two commercial or proprietary models and at least one open-weight or smaller model
- A model router that selects models using task type, capability, confidence, latency and estimated cost
- Tool and data connectors for search, retrieval, APIs, databases, code execution or geospatial operations
- Structured outputs that follow a defined schema and can be consumed by the application
- A verifier or critic that checks important facts, calculations and model outputs
- Fallback and recovery when a selected model, tool or data source is unavailable or returns a low-confidence result
- Shared state or memory that records the plan, intermediate results, sources and decisions

Choose One Implementation Use Case

> **Implementation choice:** Implement the generic architecture for any one of the preferred use cases below. Teams may propose another meaningful use case of their own, but the following FarmwiseAI-related use cases are preferable.

- Agriculture advisory and intelligence using crop photographs, weather, advisory documents and parcel or crop data
- Land-document and parcel intelligence using scanned documents, cadastral maps, databases and spatial constraints
- District or departmental intelligence using multiple applications, reports, datasets and news sources
- Street-level urban asset or property intelligence using imagery, OCR, maps and reference records

What the Team Is Expected to Build

The final prototype should be one working application for the selected use case. A user gives a real request together with the relevant files, images or data. The system must decide how to complete the request, use different models for different parts of the work, call the required tools, verify the result and present one useful final output. It should demonstrate genuine orchestration rather than sending the same prompt independently to several models.

Common End-to-End Behaviour

- Interpret the request and identify the input types, required outputs, constraints and missing information.
- Create an explicit execution plan and select the models, tools and data sources required for each step.
- Use the open-weight or smaller model wherever it can complete a step reliably and economically.
- Use a commercial or proprietary model for complex planning, multimodal understanding, verification or an uncertain step where justified.
- Invoke the required tools or data connectors and preserve source references for every important result.
- Merge intermediate outputs into the selected use case's final response, interface, map, report or structured record.
- Verify critical outputs and demonstrate at least one fallback, retry, alternate-model or human-review path.

Detailed Illustration: Parcel-Level Crop Intelligence and Advisory

> **Illustration only:** This example explains the expected depth and clarity. Teams choosing another preferred use case should demonstrate the same planning, routing, tool use, verification and final-product behaviour in their own context.

- **Example User Query**

> *"What crop is planted in Survey Number 202/55 in the selected village? Identify the parcel and its owner, retrieve the field photograph captured for that parcel, predict the crop and its current growth stage, and recommend the appropriate fertilizer and crop-management practices using the available advisory data.*

- **How the Models and Agents May Work Together**

- The planner identifies the village, Survey Number 202/55, the required parcel, ownership, image, crop-prediction and advisory outputs, and creates the execution plan.
- A geography agent resolves the district, taluk and village hierarchy, locates the cadastral parcel and confirms that the survey number is mapped to the correct geometry.
- A land-record connector retrieves the parcel attributes and owner name from the relevant registry or database and preserves the source record.

- An image-retrieval agent uses the parcel identifier, survey number, location and capture metadata to retrieve the field photograph associated with that parcel.
- An open-source or smaller vision model performs the first crop, growth-stage and crop-condition prediction and returns structured labels with confidence.
- Low-confidence, conflicting or complex images are routed to one or more proprietary vision-language models. The orchestrator compares the results and selects or verifies the final prediction.
- A knowledge agent retrieves stage-specific fertilizer recommendations and crop-management practices from the approved agronomy, soil, weather and advisory data sources.
- A verifier checks the parcel-owner match, image-to-parcel relationship, crop and stage prediction, and the applicability and source of each recommendation.
- The application combines the verified geography, ownership, image analysis and advisory into one traceable result for the user.

- **Expected Result for This Illustration**

- A map highlighting Survey Number 202/55 in the correct village, together with its parcel and administrative details
- The parcel identifier, land-record source and owner name retrieved from the relevant database
- The field photograph associated with the parcel and its capture metadata
- Predicted crop, growth stage and supported crop condition or disease, with confidence and model-verification status
- Stage-appropriate fertilizer and crop-management recommendations with supporting advisory sources
- A route trace showing every agent, data source, tool and model used, including the open-source and proprietary model decisions, latency and estimated cost

Other Preferred Implementation Examples

- **Land Document and Parcel Intelligence**

> *Read the supplied land document, extract survey numbers, owners and extents, match them with cadastral parcels and database records, identify discrepancies and present the verified result on a map and in a report.*

Expected outcome: extracted structured records, document-to-parcel matches, discrepancy categories, source evidence, confidence and a map-based result. The system should route OCR, document understanding, geospatial matching and verification to suitable models or tools.

- **District and Departmental Intelligence**

> *Search the connected district applications, reports and approved news sources, identify important incidents, group duplicate reports and prepare a source-linked Collector briefing with an action list.*

Expected outcome: a verified cross-source summary, incident categories and locations, duplicate clusters, evidence links and a briefing or dashboard. The system should demonstrate planning, source selection, extraction, verification and fallback.

- **Street-Level Urban Asset Intelligence**

> *Analyse the selected Street View locations, identify buildings and public assets, extract visible signboards, match the findings with reference records and show unmatched or uncertain observations.*

Expected outcome: structured detections, OCR results, street-level locations, matched and unmatched records, confidence and evidence. Smaller vision models should handle routine cases, with selective proprietary-model escalation for complex scenes.

Expected Demonstration

> **Evaluation focus:** Task 1 primarily evaluates proficiency in LLMs and agentic frameworks: how well the team plans a complex request, handles multimodal inputs, orchestrates multiple agents, routes work across proprietary and open-source models, calls tools and data sources, preserves state, verifies results and manages confidence, fallback, latency and cost.

- The original request, generated plan and progress of every step
- Which model or tool handled each step and why it was selected
- Structured intermediate and final outputs with source traceability
- Confidence, latency and estimated cost for the route used
- A visible verification result and one demonstrated fallback or recovery path
- A complete end-to-end result for the chosen use case and a comparable unseen request

Capabilities the Final Prototype Must Demonstrate

- At least three models in total: two commercial or proprietary models and one open-weight or smaller model
- A meaningful and justified role for every model rather than sending the same prompt to several models
- An agentic framework demonstrating planning, orchestration, model routing, tool calling, shared state and multimodal input handling
- At least two meaningful tool or data-source calls
- Structured outputs that drive the selected application's result or interface
- Verification, confidence handling and a demonstrated fallback path
- Model-route, latency and cost comparison
- One complete implementation use case without hard-coded answers

TASK 2

Multimodal GeoAI and Land Document Intelligence Platform

Connect land documents, databases and geospatial layers in a synchronized map-based workspace.

Objective

Build a GeoAI platform that extracts information from land documents, matches it with survey numbers, subdivisions, ownership records and cadastral parcels, and allows users to create maps, dashboards, tables and reports using natural language.

Relevant Data Sources

- Cadastral parcel, survey subdivision, village and administrative boundary layers
- Patta, land record, acquisition notification, award and compensation documents
- Ownership, survey-number and acquisition-stage databases
- Road, power, waterbody, flood-risk and restricted-area layers
- Satellite or drone imagery

Key Relationship Requirement

The system must maintain relationships between documents, survey numbers, subdivisions, owners, parcel boundaries, land extent, acquisition stage, compensation status and geographic constraints. It should not treat each document or dataset as an isolated file.

Product Interface Expectations for This Challenge

> **Task-specific expectation:** The proposed solution should be an AI-first, map-based product workspace rather than only an answer inside a chat window. A natural-language request should create or update the maps, dashboard components, document views, tables and reports required for that request.

- Use information matched across multiple documents, databases and geospatial layers.
- Keep the document, map, KPI cards, charts and tables synchronized.
- Selecting a document reference, parcel or chart segment should filter and update the connected views.
- Allow the user to modify, save, reopen and export the generated map or dashboard workspace.
- Show the source, assumption, confidence and verification status behind important results.
- Provide understandable legends, filters, loading states and error messages.

- Do not hard-code the example queries or their answers.

Detailed Example A: Land Document and Parcel Matching

- **Example User Query**

> *Read the uploaded land-acquisition notification and award statement. Extract all survey numbers, subdivisions, owner names and land extents. Match them with the cadastral parcels and ownership database. Show unmatched records and extent differences on the map and create a land-acquisition progress dashboard.*

- **Expected System Behaviour**

- Process scanned or digital documents and extract survey numbers, owners, extents and document references.
- Resolve survey-number and subdivision variations across the documents and database.
- Locate the corresponding cadastral parcels.
- Compare document extent with GIS extent and document owner with registry owner.
- Identify records missing from documents, the database or the parcel map.
- Determine the acquisition stage and related documents for each parcel.
- Generate the discrepancy layer, progress dashboard and report.

- **Expected Result on the Screen**

- Synchronized document and map view: clicking a survey number in a document highlights the parcel; clicking a parcel opens its related records and documents.
- Map: matched parcels, unmatched parcels, extent differences, ownership differences and records missing from documents or GIS.
- KPI cards: total surveys, total extent, matched records, unmatched records, extent differences, ownership differences and stage counts.
- Charts: land extent by acquisition stage, match status, ownership category and compensation status.
- Discrepancy table: village, survey/subdivision, owners, document/GIS extents, difference, status, stage and source document.

Detailed Example B: Land Suitability Intelligence

- **Example User Query**

> *Identify government-owned parcels larger than 20 acres within five kilometres of a major road. Exclude parcels affected by waterbodies, flood-risk zones or protected areas. Rank the suitable parcels based on road and power accessibility and create a comparison dashboard.*

- **Expected Result on the Screen**
- Map: suitable and excluded parcels, roads, power infrastructure, waterbodies, flood zones and protected areas.
- KPI cards: parcels analysed, suitable parcels, suitable area and exclusion counts by reason.
- Charts: suitable area by village, suitability category, exclusion reason and accessibility comparison.
- Table: parcel, survey number, village, ownership, area, distances, restrictions, suitability score and reason.

Comparable Queries the Product Must Handle

> *Show survey numbers present in the document but missing from the parcel map.*

> *Highlight parcels where document extent and GIS extent differ by more than five percent.*

> *Show all documents related to the selected parcel and create a village-wise discrepancy report.*

> *Create a dashboard showing acquisition progress by village and add an extent-by-stage chart.*

> *Show only government parcels affected by both flood risk and waterbody boundaries.*

Capabilities the Final Prototype Must Demonstrate

- At least three data types, including one land document and one spatial layer
- Automatic document extraction and document-to-parcel matching
- One attribute query and one multi-layer spatial query
- Visible SQL, code or GIS operation generation
- Synchronized document, map, chart, table and dashboard components
- Natural-language modification, saving and export
- Evidence, assumptions and traceability

TASK 3

Satellite-Embedding-Based Crop Classification

Build parcel-level crop classifiers from supplied geospatial embedding features rather than processing raw satellite imagery from scratch.

Objective

Build a crop-classification system using supplied Google Satellite Embedding features, Presto remote-sensing embeddings or both. The team should link the embedding vectors with agricultural parcels and verified crop labels, train a suitable classification head or lightweight model, predict the cultivated crop for each parcel and report confidence and validation performance.

Data That May Be Provided

- Google Satellite Embedding feature vectors or tiles for the selected study area
- Presto embeddings representing multisensor remote-sensing time-series information
- Agricultural parcel and administrative boundary maps
- Crop labels for a training subset and a separate held-out evaluation set
- Optional crop-season, weather or registry information

> **Remote-sensing knowledge:** Basic knowledge of remote-sensing data is useful for understanding pixels, parcels, spatial resolution, seasonal patterns and geographic leakage. Teams are not expected to create the satellite embeddings or perform a complete raw-image preprocessing workflow unless they specifically propose it.

Capabilities the Prototype Must Develop

- Inspect the supplied embedding features and correctly associate them with parcels and crop labels
- Develop an appropriate spatial or temporal aggregation method when multiple embedding observations represent one parcel
- Train and compare suitable classifiers or lightweight neural-network heads using the supplied embeddings
- Predict crop class, probability and confidence for every evaluation parcel
- Identify low-confidence or unusual embedding patterns for field verification
- Validate performance without parcel, neighbourhood, geography or season leakage

Expected Technical Workflow

- Understand the supplied embedding format, coordinate reference, dates or seasons, parcel boundaries and crop labels.
- Link the Google Satellite Embedding or Presto features to the correct parcels and calculate justified parcel-level representations.
- Explore class separability, feature quality, missing observations and class imbalance.
- Compare one or more suitable approaches such as gradient-boosted trees, random forests, linear classifiers, multilayer perceptrons or another justified model head.
- Tune the selected model using only the training and validation data and document the experiment design.
- Predict crop class and confidence for the held-out parcels and produce parcel-level output files.
- Report overall and per-class performance and explain the most important error patterns.

Example User Query

> *Using the supplied satellite embeddings, classify all agricultural parcels in the selected study area. Show crop-wise cultivated area and confidence, and highlight low-confidence parcels that require field verification.*

Expected Result

- Parcel-level output containing parcel identifier, predicted crop, probability or confidence and review status
- Map or analytical view showing crop classes and low-confidence parcels
- Crop-wise area and class-distribution summary
- Comparison of the evaluated embedding source or model-head approaches
- Validation report containing overall accuracy, macro F1, per-class precision/recall/F1 and confusion matrix

> **Performance target:** The target is more than 95% accuracy on the held-out evaluation set for the supported crop classes. Per-class metrics must also be reported, and spatial or parcel leakage between training and evaluation data will invalidate the claimed result.

Comparable Analysis Requests

> *Show crop-wise area for the selected village and highlight parcels below 80% confidence.*

> *Compare the performance obtained from Google Satellite Embeddings and Presto embeddings where both are supplied.*

> *Show villages where predicted crop area differs significantly from the supplied registry.*

> *List the crop classes with the highest confusion and show representative errors.*

> *Export the classified parcel layer, confidence values and crop-area summary.*

Capabilities the Final Prototype Must Demonstrate

- Reproducible embedding-to-label training, validation and inference pipeline
- Correct parcel and label linkage with documented spatial leakage controls
- Comparison of suitable embedding sources or classification heads
- Held-out evaluation with overall and per-class metrics
- Parcel-level crop classification, confidence output and downloadable predictions

TASK 4

Mobile-Image Crop, Stage and Disease Model

Use proprietary vision models as teachers to train and validate a lower-cost open-source crop model.

Objective

Build a crop-vision model that predicts crop name, crop stage and supported crop disease or healthy condition from mobile-phone photographs. Proprietary vision or vision-language models may be used to generate candidate labels, but the final deliverable must include a trained open-source student model suitable for lower-cost inference.

Data That May Be Provided

- A set of mobile-phone crop photographs
- Partially labelled or unlabelled training images
- Supported crop-name, crop-stage and disease categories
- Optional location, date and crop metadata
- A manually verified held-out test set

Capabilities the Prototype Must Develop

- Generate structured candidate labels from one or more approved proprietary teacher models
- Filter labels through confidence, consensus and human-review rules
- Create a cleaned, versioned and class-balanced training dataset
- Fine-tune a selected open-source vision or vision-language model
- Predict crop, stage, disease or healthy condition with confidence and abstention for unsupported cases
- Automatically escalate low-confidence, out-of-distribution or conflicting student predictions to a proprietary teacher model
- Add verified corrections to a feedback dataset and periodically improve the open-source model
- Compare teacher and student models for accuracy, latency, model size and inference cost
- Provide reproducible individual-image and batch-inference workflows

Required Teacher-Student Workflow

- Send the training photographs to one or more approved commercial cloud vision or vision-language services.

- Request crop name, crop stage, disease or healthy condition, confidence and short visual evidence in a structured format.
- Resolve conflicting labels using confidence thresholds, model consensus or another documented approach.
- Send ambiguous and low-confidence images to a human-review queue.
- Create the verified training dataset and document class balance and label provenance.
- Fine-tune MobileNet, EfficientNet, ConvNeXt, ViT, CLIP/SigLIP-based classifier or another justified open-source model.
- Evaluate only on the verified held-out test set and compare the student with the teacher models.

Required Continuous-Learning Loop

- Run each new image through the deployed open-source student model first.
- Accept the student result when it meets the defined confidence and validity rules.
- Automatically send low-confidence, unsupported, out-of-distribution or conflicting cases to an approved proprietary teacher model.
- Verify the teacher response automatically where possible and send genuinely ambiguous cases for human review.
- Store the image, student prediction, teacher response, verification decision and final label in a versioned feedback dataset.
- Periodically retrain or fine-tune a candidate student model with the new verified cases.
- Compare the candidate against the current production model on the held-out test set before promoting the new version.

Example User Query

> *Analyse the uploaded crop photograph. Identify the crop, estimate its growth stage, detect whether it is healthy or affected by a supported disease, and show confidence, visual evidence and the recommended next action.*

Expected Model Output

- Uploaded photograph with relevant crop or symptom region highlighted where possible
- Predicted crop name, stage, health or disease class and confidence
- Alternative predictions or human-review status when confidence is low
- Similar verified training images and teacher-label provenance
- Structured individual and batch prediction files
- Evaluation summary comparing accuracy, latency, model size and cost between teacher and student models

Performance and Validation Requirement

> **Accuracy target:** The trained open-source model should target more than 95% accuracy on the verified held-out test set for the supported classes. Crop name, crop stage and disease must be evaluated separately using per-class precision, recall, F1, macro F1 and confusion matrices.

- Near-duplicate images or images from the same field burst must not be split across training and testing.
- Teacher-generated labels must not be used as ground truth for the held-out test set.
- Unsupported or ambiguous images should return low confidence or human-review status rather than a forced prediction.

Comparable Inference and Evaluation Scenarios

> *Classify this batch of photographs and return crop, stage, disease and confidence for every image.*

> *Show only images where the teacher and student models disagree.*

> *Display low-confidence disease predictions and create a review queue.*

> *Compare accuracy, latency, model size and cost across the evaluated models.*

> *Export the verified predictions, evaluation metrics and model-performance report.*

Capabilities the Final Prototype Must Demonstrate

- Reproducible teacher-labelling, data-cleaning, training and inference workflow
- Validated open-source student model with separate task metrics
- Individual-image and batch-inference pipeline
- Automated low-confidence teacher fallback, disagreement handling and human-review workflow
- Versioned feedback data, retraining or fine-tuning and controlled model promotion
- Teacher-student cost, latency, size and accuracy comparison

TASK 5

Google Street View-Based Urban Asset and Property Intelligence

Use Google Street View 360-degree imagery to create a street-level inventory of buildings, properties and public assets.

Objective

Build a computer-vision and GeoAI product that uses Google Street View 360-degree panoramas to understand buildings, properties, streets and public infrastructure. The solution should demonstrate an innovative and compliant use of Google Street View APIs, services or permitted embedded Street View experiences, convert the visual observations into structured information and present street-level findings on a map.

Relevant Data Sources

- Google Street View 360-degree panoramas and panorama metadata for selected streets or corridors
- Street View Static API, Maps JavaScript Street View services or permitted embedded Street View views
- Street, road, building-footprint and property-reference layers
- Existing streetlight, electric-pole or infrastructure inventory
- Signboard, business, building or property reference records

> **Access and compliance:** The prototype must use only approved access methods, credentials and permitted Street View content. Teams must comply with the applicable Google Maps Platform terms, attribution and content-display requirements and must not bypass access controls or extract imagery through prohibited scraping.

Expected Street-Level Intelligence

- Building use, number of visible floors, condition, frontage and selected property characteristics
- Commercial signboards, building names and visible property identifiers through OCR
- Streetlights, electric poles, transformers, traffic signs, waste bins and other selected urban assets
- Street or approximate asset location, confidence, panorama evidence and human-review status
- Matching with existing property and infrastructure records

Cost-Aware 360-Degree Vision Workflow

The application should avoid processing every panorama view with a large vision-language model. It should select useful panorama locations and headings, use smaller detection, classification and OCR models first, and invoke a larger VLM only for uncertain or complex scenes. The interface should show the processing route, latency and estimated cost.

Detailed Example: Street-Level Urban Asset and Property Intelligence

- **Example User Query**

> *Analyse the selected streets using Google Street View. Identify buildings, estimate visible floors and building use, extract commercial signboards, detect streetlights and electric poles, and match the observations with the property and infrastructure records. Show missing, unmatched and low-confidence results on the map.*

- **Expected System Behaviour**
  - Discover available Street View panorama coverage and metadata for the selected streets or corridor.
  - Select useful panorama locations, headings and fields of view through an approved Street View interface.
  - Detect buildings and selected public assets using smaller vision models and run OCR for visible identifiers and signboards.
  - Invoke a larger vision-language model only for low-confidence building use, floor count or complex scenes.
  - Resolve repeated observations of the same building or asset across nearby panorama positions and headings.
  - Associate each observation with a street, building footprint or approximate map location.
  - Match observations against existing property and infrastructure records.
  - Generate the street-level map, dashboard, discrepancy table, evidence view and review queue.

- **Expected Result on the Screen**
  - Street-level map: analysed streets, panorama locations, buildings and assets by type, match status and confidence.
  - KPI cards: streets covered, buildings analysed, unmatched properties, streetlights, poles and low-confidence observations.
  - Charts: building use, floor distribution, asset type, database-match status and street distribution.
  - Table: asset or building identifier, street, approximate location, detected attributes, OCR text, confidence, matched record and review status.
  - Evidence panel: permitted Street View reference or displayed panorama context, selected heading, detected region, model route and verification status.
  - Cost panel: views handled by small models, views escalated to a VLM, latency and estimated cost.

Interaction: Selecting a street or unmatched-property segment should filter the map and results. Selecting a mapped observation should open its permitted Street View context, detected attributes, OCR output and matching reference record.

Comparable Queries the Product Must Handle

> *Show commercial buildings with more than two visible floors that do not have a matching property record.*

> *Show selected streets where no streetlight is detected within the specified interval.*

> *Display only low-confidence floor-count predictions and create a review queue.*

> *Add a chart showing unmatched buildings by street and zoom the map when a bar is selected.*

> *Compare the cost and accuracy of processing every selected view with a VLM against the proposed routed workflow.*

Capabilities the Final Prototype Must Demonstrate

- An innovative, compliant Google Street View API, service or embedded-view integration
- Selection and handling of useful views from 360-degree panoramas
- Detection of selected building attributes and at least two urban-asset types
- OCR, structured extraction, street-level geospatial association and duplicate handling
- Matching against at least one property or infrastructure reference dataset
- Small-model-first routing with selective VLM escalation
- Synchronized street map, evidence, charts, table and review queue with accuracy, latency and cost comparison

TASK 6

Collector's Unified Application, Web and District Intelligence System

Create one search, monitoring and briefing layer across authorized applications, websites and district news sources.

Objective

A District Collector may need to use several web or mobile applications, separate logins, departmental dashboards, government websites and news sources to understand what is happening in the district. Build an AI-enabled system that connects these authorized sources, searches and collects the required information, resolves it to a common district geography and produces one traceable operational view and briefing workspace.

> **Domain knowledge:** Participants are not expected to know district, taluk, block or village structures before submitting the proposal. For prototype development, FarmwiseAI will provide the relevant administrative hierarchy, boundary maps, lookup tables, data dictionaries and domain explanations needed for the approved implementation.

Sources That May Be Connected

- Authorized departmental web or mobile applications, APIs, database exports and reports
- Approved test accounts or mock applications representing systems with separate logins
- District columns, district pages and local reports from multiple newspapers or news websites
- Government websites, public notices, press releases, grievance feeds and incident reports
- Agriculture, water, land, disaster, infrastructure or other departmental datasets
- District, taluk, block, village and other relevant boundary maps and lookup tables

> **Authorized access only:** The prototype must use approved credentials, APIs, exports, connectors or permitted web extraction and browser automation. It must not bypass authentication, access controls, paywalls, robots restrictions or website terms. Mock applications or sample exports may be used where live access cannot be provided.

Capabilities the Prototype Must Develop

- Connect and search across multiple authorized applications, websites, documents and structured datasets from one interface

- Use APIs, connectors or permitted browser automation to collect and refresh information from the approved sources
- Extract district events, incidents, locations, dates, departments, people, organisations, severity and current status
- Resolve inconsistent district, taluk, block and village names using the supplied administrative hierarchy
- Deduplicate repeated reports, cluster related stories or records and maintain a time-ordered incident history
- Generate daily, weekly, monthly and quarterly summaries, trends, alerts and department-wise action lists
- Support natural-language search and automatically create or modify maps, KPI cards, charts, tables and reports
- Preserve source links, timestamps, evidence, confidence and an audit history for every important finding

Detailed Example: Collector's District News and Incident Briefing

- **Example User Query**

> *Prepare today's Collector briefing by reviewing the connected departmental applications and the district columns of all approved newspapers. Group duplicate reports, identify the important incidents in the district, locate them on the map, explain why they require attention and create a department-wise action list. Also show the weekly and monthly trend for each incident category.*

- **Expected System Behaviour**
  - Access each approved source through its API, connector, export or permitted authenticated workflow.
  - Collect the relevant records, district news articles, notices and reports for the requested period.
  - Extract incident type, location, date, affected people or assets, department, severity and source evidence.
  - Resolve place names to the correct district, taluk, block or village and flag unresolved locations for review.
  - Detect duplicates and cluster reports that describe the same event across applications or newspapers.
  - Rank important incidents using documented priority rules and generate a concise, source-linked summary.
  - Create the district map, incident timeline, KPI cards, category trends, source view and department-wise action list.
  - Save the briefing and make the same pipeline available for daily, weekly, monthly and quarterly reporting.

- **Expected Result on the Screen**
  - Unified search box covering the connected applications, websites, reports and news sources
  - Collector briefing containing major incidents, why each matters, source evidence and proposed follow-up
  - District map showing incident locations with taluk or village drill-down where the data permits
  - Timeline and charts comparing incident categories, severity, source, geography and reporting period

- Incident table containing event, location, date, category, severity, department, source count and action status
- Source and evidence panel showing related application records, reports and newspaper links for the selected incident
- Controls for daily, weekly, monthly and quarterly briefings and downloadable reports

> **Interaction:** Selecting an incident or chart category should filter the map, sources and action list. The user should be able to open the supporting evidence, change the reporting period and save the configured briefing as a reusable Collector workspace.

Comparable Natural-Language Requests

> “ *Give me the weekly summary of road accidents, flooding and public-infrastructure complaints in the district.*

> “ *Show incidents reported by more than one newspaper but missing from the connected departmental application.*

> “ *List the taluks with the highest number of unresolved incidents during the last month.*

> “ *Create a quarterly trend of incident categories and prepare a department-wise follow-up report.*

> “ *Show every source and application record used to create this briefing.*

> “ *Save this workspace as Collector's Daily Briefing and reopen last week's version.*

Optional Voice Interaction

Voice interaction is an optional additional capability. Teams may allow the same unified search, briefing and dashboard commands to be spoken instead of typed. The core application will be considered complete if all required functions work through text-based natural-language interaction.

> “ *Prepare today's district briefing, show the three most serious incidents and read the action summary.*

Capabilities the Final Prototype Must Demonstrate

- Integration of at least two authorized applications, websites, news sources or datasets
- A unified natural-language search and briefing experience across the connected sources
- Permitted collection or automation workflow with refresh, error and login-session handling
- District entity and location extraction, deduplication, incident clustering and timeline creation
- Daily, weekly, monthly or quarterly summaries with source-linked incidents and trends
- Synchronized map, KPI, chart, table, evidence and action-list components
- Source traceability, audit history and saved workspace or exported report
- Optional voice commands, if implemented

After Shortlisting: Prototype Development Stage

FarmwiseAI will shortlist participants based on the quality, clarity, innovation and feasibility of the submitted proposals. Only the shortlisted teams will proceed to the prototype-development stage.

Data, Infrastructure and Technical Inputs

- The maps, administrative boundaries, documents, datasets, imagery, database tables, schemas and data dictionaries needed for the approved prototype
- Sample APIs or mock interfaces when direct access to an application is not available
- Explanations of domain terms, data relationships and the capabilities expected from the selected challenge
- Technical guidance from the FarmwiseAI product, GIS, data science and engineering teams
- AWS cloud credits and LLM or model API credits, based on the approved proposal and demonstrated technical requirement

The final data package, infrastructure and credits will be decided after the proposal review. The proposal should therefore state clearly what data, compute and model access the team expects to require.

Prototype Expectation

> **Development period:** Shortlisted teams will be expected to develop a complete working prototype within approximately two to three weeks, following the project plan presented in the approved proposal.

The final demonstration should show the core capabilities promised in the proposal and specified in the selected challenge. The expected output may be a model, analytical workflow, map-based application, dashboard or integrated AI product, depending on the chosen task.

Grow Smarter

FarmwiseAI

We look forward to your proposals.

Pick your challenge, build your team and show us what you can create.

Ready to take on a challenge?

Register your team and pick your task on the enrollment platform.

CLICK HERE TO APPLY →